# Les complexités en temps

Projet ft_turing, bonus complexité. Document à compléter au fur et à mesure : pour l'instant **O(n)** et **O($n^2$)**. Code à jour : commit `63e24f9` (branche complexity).

**L'idée de départ.** On ne mesure pas une machine sur une seule entrée. On la lance sur **plusieurs entrées de tailles différentes** et on regarde **comment le nombre de pas grandit** quand l'entrée grandit.

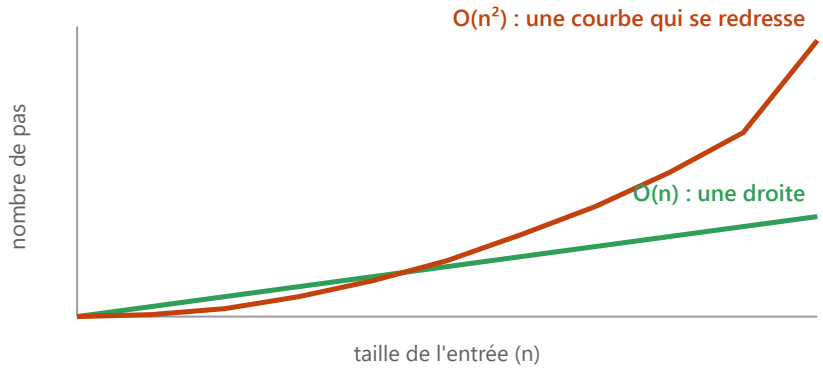
La complexité, c'est le **tarif** de la machine, pas le prix d'une course : « combien ça coûte en plus quand le ruban s'allonge ».

### Vocabulaire

- **n** : la taille de l'entrée, c'est-à-dire le nombre de cases du ruban au départ. `1+1=` donne n = 4.
- **Un pas** : une instruction de la machine (lire, écrire, se déplacer). C'est une ligne de la trace, et c'est ce que compte `count`.
- **O(...)** : se lit « grand O de... ». C'est le nom de la courbe que suit le nombre de pas.

### Le test le plus simple : doubler l'entrée

| Quand n double... | les pas sont multipliés par | Complexité |
|---|---|---|
| 2 fois plus de cases | × 2 | linéaire, O(n) |
| 2 fois plus de cases | × 4 | quadratique, O($n^2$) |




## Complexité linéaire O(n)

**En une phrase :** chaque case en plus coûte **toujours le même nombre de pas**.

### L'image : le taxi

Le compteur affiche toujours le même prix au kilomètre. 2 fois plus de kilomètres, c'est 2 fois plus cher. Le kilomètre n°100 coûte autant que le kilomètre n°1.

### Exemple : `unary_add`

La machine passe **une seule fois** sur chaque case, de gauche à droite, sans revenir en arrière :

```
[1+1=] (scanright, 1) -> RIGHT     case 1
[1+1=] (scanright, +) -> RIGHT     case 2 (le + devient 1)
[111=] (scanright, 1) -> RIGHT     case 3
[111=] (scanright, =) -> LEFT      case 4 (le = est effacé)
[111.] (eraseone, 1)  -> HALT      on efface le dernier 1
```

Une case en plus ajoute une ligne, donc un pas. Au total, **pas = n + 2** : les 2 pas en plus sont ceux de la fin, qui ne dépendent pas de la taille.

### Les vrais chiffres (programme ft_turing)

| n | pas | pas en plus ÷ cases en plus |
|---|---|---|
| 4 | 6 | – |
| 6 | 8 | (8 − 6) ÷ (6 − 4) = **1** |
| 10 | 12 | (12 − 8) ÷ (10 − 6) = **1** |
| 18 | 20 | (20 − 12) ÷ (18 − 10) = **1** |
| 34 | 36 | (36 − 20) ÷ (34 − 18) = **1** |
| 66 | 68 | (68 − 36) ÷ (66 − 34) = **1** |

De 4 à 66 cases, chaque case en plus coûte **toujours 1 pas** : c'est linéaire.

### La méthode de calcul

$$(pas2 - pas1) \div (n2 - n1)$$

On le calcule pour chaque paire d'entrées voisines (liste triée par taille). **Si le résultat est le même pour toutes les paires, c'est O(n).**

> **Pourquoi pas simplement pas ÷ n ?** Avec pas = n + 2, on obtient 6 ÷ 4 = 1,50, puis 8 ÷ 6 = 1,33, puis 10 ÷ 8 = 1,25. Le « +2 » fausse le rapport sur les petites entrées. En **soustrayant** deux entrées, le +2 disparaît : il reste exactement 1.

Le code est dans la partie « Le code » plus loin : c'est la ligne `| Linear -> longSize -. shortSize`.

Machines du sujet en O(n) : `unary_add` et `02n` ( `even` ).

## Complexité quadratique $O(n^2)$

**En une phrase :** chaque case en plus coûte **de plus en plus cher**, mais ce coût **augmente toujours de la même façon**.

« Quadratique » vient de « carré » : le nombre de pas grandit comme **n × n**.

### L'image : l'escalier

À chaque marche, tu montes plus haut que la précédente, mais **toutes les marches ont la même hauteur**. Le coût d'une case en plus, c'est la hauteur où tu es : elle monte toujours. L'augmentation, c'est la hauteur d'une marche : elle reste la même.

| Case en plus | 1re | 2e | 3e | 4e | 5e |
|---|---|---|---|---|---|
| elle coûte (pas) | 3 | 5 | 7 | 9 | 11 |
| de plus que la précédente | – | **+2** | **+2** | **+2** | **+2** |

Exemple d'une machine idéale qui fait exactement $n^2$ pas : 1, 4, 9, 16, 25, 36…

### Pourquoi le palindrome est quadratique

Pour vérifier `10101`, la machine :

1. lit la 1re case, puis **traverse tout le ruban** pour voir la dernière ;
2. revient au début, lit la 2e case, retraverse pour voir l'avant-dernière ;
3. et ainsi de suite.

Elle fait environ **n allers-retours**, et chaque aller-retour fait environ **n pas** : au total, **n × n**. Plus le ruban est long, plus chaque aller-retour est long : c'est pour ça que chaque case en plus coûte plus cher.

Comparaison : `unary_add` ne traverse le ruban qu'**une fois**, donc il est linéaire.

### Ce que donne la méthode O(n) sur une machine en $n^2$

**Divisé par les cases en plus**

| n | pas | ÷ cases en plus |
|---|---|---|
| 2 | 4 | – |
| 4 | 16 | 12 ÷ 2 = **6** |
| 8 | 64 | 48 ÷ 4 = **12** |

Ça **grandit** : ce n'est pas linéaire.

**Divisé par les carrés en plus**

| n | $n^2$ | ÷ carrés en plus |
|---|---|---|
| 2 | 4 | – |
| 4 | 16 | 12 ÷ 12 = **1** |
| 8 | 64 | 48 ÷ 48 = **1** |

C'est **constant** : c'est quadratique.

### La méthode de calcul

$$(pas2 - pas1) \div (n2^2 - n1^2)$$

C'est la même formule que pour O(n), avec les tailles **au carré**. **Si le résultat est le même pour toutes les paires, c'est O($n^2$).**

### Les vrais chiffres : `palindrome`

| n | pas | ÷ cases en plus (test O(n)) | ÷ carrés en plus (test O($n^2$)) |
|---|---|---|---|
| 4 | 23 | – | – |
| 8 | 59 | 36 ÷ 4 = 9 | 36 ÷ 48 = 0,75 |
| 16 | 179 | 120 ÷ 8 = 15 | 120 ÷ 192 = 0,63 |
| 32 | 611 | 432 ÷ 16 = 27 | 432 ÷ 768 = 0,56 |

Test O(n) : 9, 15, 27, ça grandit beaucoup, donc **non**. Test O($n^2$) : 0,75, 0,63, 0,56, presque constant, donc **oui**.

> **Pourquoi « presque » ?** Comme le +2 de `unary_add`, le palindrome fait aussi des pas qui suivent n (et pas $n^2$). Ils pèsent sur les petites entrées puis comptent de moins en moins. C'est pour ça que `compareRatios` compare avec une **tolérance de 20 %** plutôt qu'avec un écart fixe de 0,1 : les valeurs pour O($n^2$) sont petites, un écart de 0,1 y est énorme.

Le code est dans la partie « Le code » plus loin : c'est la ligne `| Quadratic -> (longSize *. longSize) -. (shortSize *. shortSize)`. Machines du sujet en O($n^2$) : `palindrome`, `0n1n`, et la machine universelle (allers-retours sur tout le ruban à chaque étape).

## Récapitulatif

| Complexité | Chaque case en plus coûte… | n doublé → pas × | Ce qui reste constant | Machines |
|---|---|---|---|---|
| **O(n)** | toujours pareil | 2 | (pas2 − pas1) ÷ (n2 − n1) | unary_add, even |
| **O($n^2$)** | de plus en plus, toujours du même surplus | 4 | (pas2 − pas1) ÷ ($n2^2$ − $n1^2$) | palindrome, 0n1n |

### Ce que fait `printComplexity`

1. Trier les entrées par taille (une seule fois).
2. Tester O(n) : si c'est constant, afficher `Complexity: O(n)`.
3. Sinon tester O($n^2$) : si c'est constant, afficher `Complexity: O(n^2)`.
4. Sinon afficher que la complexité est plus grande ou inconnue.

> **Limite à connaître :** avec des entrées trop courtes (n = 2, 4, 8), les pas « fixes » de la machine pèsent trop et le résultat peut être faux ou inconnu. Avec des entrées plus longues (jusqu'à 30 cases ou plus), il est juste.

## Le code ( `execution/printComplexity.ml` , commit `63e24f9` )

### 1. La liste des complexités qu'on sait tester

```
type complexity = Linear | Quadratic
```

### 2. Le calcul, pour chaque paire de rubans voisins

Le `match` choisit la formule : on applique la courbe à la taille du ruban long et à celle du court, puis on soustrait. Pour O($n^2$), on met chaque taille au carré, puis on soustrait.

```
let rec computeStepsPerUnit (curve : complexity) (tapes : (string * int) list) : float list =
    match tapes with
    | (shortTape, shortCount) :: ((longTape, longCount) :: _ as rest) ->
        let shortSize : float = Float.of_int (String.length shortTape) in
        let longSize : float = Float.of_int (String.length longTape) in
        let extraSteps : float = Float.of_int (longCount - shortCount) in
        let extraSize : float =
            match curve with
            | Linear    -> longSize -. shortSize
            | Quadratic -> (longSize *. longSize) -. (shortSize *. shortSize)
        in
        let stepsPerUnit : float = extraSteps /. extraSize in
            stepsPerUnit :: computeStepsPerUnit curve rest
    | _ -> []
```

### 3. Les vérifications

`compareRatios` vérifie que deux valeurs voisines sont à moins de 20 % l'une de l'autre ( `0.2 * max r1 r2` ). Chaque `check_…` calcule la liste avec sa courbe, puis appelle `compareRatios` .

```
let rec compareRatios (ratios : float list) : bool =
    match ratios with
    | r1 :: r2 :: rest ->
        if Float.(abs (r1 - r2) <= 0.2 * max r1 r2) then
            compareRatios (r2 :: rest)
        else
            false
    | _ -> true

let check_Linear_time_complexity (sortedTapes : (string * int) list) : bool =
    let stepsPerCellList : float list = computeStepsPerUnit Linear sortedTapes in
        compareRatios stepsPerCellList

let check_Quadratic_time_complexity (sortedTapes : (string * int) list) : bool =
    let stepsPerSquareList : float list = computeStepsPerUnit Quadratic sortedTapes in
        compareRatios stepsPerSquareList
```

### 4. La fonction principale

Elle trie une seule fois, essaie O(n), puis O($n^2$). On garde la première courbe qui colle.

```
let printComplexity (tapes : (string * int) list) : unit =
    let sortedTapes : ((string * int) list) = List.sort tapes ~compare: (fun ((tape1 : string), (_ :
int)) ((tape2 : string), (_ : int))
    ->
        Int.compare (String.length tape1) (String.length tape2)
    )
    in
        if check_Linear_time_complexity sortedTapes then
            print_endline "Complexity: O(n)"
        else if check_Quadratic_time_complexity sortedTapes then
            print_endline "Complexity: O(n^2)"
        else
            print_endline "Complexity: greater than O(n^2) or unknown"
```

## Comment ajouter une complexité

Exemple avec **O(n log n)**. Toujours les mêmes 4 étapes :

**1. Ajouter le nom dans le type**

```
type complexity = Linear | Quadratic | NLogN
```

**2. Ajouter la ligne dans le `match` de `computeStepsPerUnit`** : la formule appliquée au ruban long, moins la formule appliquée au court. Si tu l'oublies, OCaml te prévient à la compilation.

```
| NLogN -> (longSize *. Float.log longSize) -. (shortSize *. Float.log shortSize)
```

**3. Ajouter un `check_...`** (copie de celui de O(n), avec le nouveau nom)

```
let check_NLogN_time_complexity (sortedTapes : (string * int) list) : bool =
    let stepsPerUnitList : float list = computeStepsPerUnit NLogN sortedTapes in
        compareRatios stepsPerUnitList
```

**4. L'ajouter dans `printComplexity`**, à sa place dans l'ordre du graphique (de la plus lente à la plus rapide) :

```
O(1)  →  O(log n)  →  O(n)  →  O(n log n)  →  O(n²)  →  O(2ⁿ)  →  O(n!)
```

```
        if check_Linear_time_complexity sortedTapes then
            print_endline "Complexity: O(n)"
        else if check_NLogN_time_complexity sortedTapes then
            print_endline "Complexity: O(n log n)"
        else if check_Quadratic_time_complexity sortedTapes then
            ...
```

### Les formules pour le `match`

| Complexité | Ligne du match |
|---|---|
| O(log n) | `(Float.log longSize) -. (Float.log shortSize)` |
| O(n log n) | `(longSize *. Float.log longSize) -. (shortSize *. Float.log shortSize)` |
| $O(n^3)$ | `(longSize *. longSize *. longSize) -. (shortSize *. shortSize *. shortSize)` |
| $O(2^n)$ | `(2.0 ** longSize) -. (2.0 ** shortSize)` |

**À savoir :**

- **O(1)** ne passe pas par ce calcul : la courbe ne bouge pas, on diviserait par 0. On vérifie simplement que tous les `count` sont égaux.
- **O(n!)** demande une petite fonction `factorial` en float. Au-delà de n = 170 environ, le float déborde.
- **Tester avec des entrées longues** (30 cases ou plus) : sur de petites entrées, n log n ressemble à n.
- Aucune machine du sujet n'est en log n, n log n, $2^n$ ou n! : on peut vérifier que ça compile, mais pas que la réponse est juste, sauf avec une petite machine de test perso (à ne pas rendre).

## À venir

Les autres courbes du graphique du sujet (page 11) seront ajoutées ici au fur et à mesure : O(1), O(log n), O(n log n), O($2^n$), O(n!).